Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

«Введение. Системный таймер»

по дисциплине «Операционные системы»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Югай П.Э

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3

Ход работы 3-7

1. Теоретическая часть 3-6

1.1 Что такое системный таймер 3

1.2 Функции системного таймера в pintOS 4-5

1.3 Блок-схема алгоритма работы функции timer(sleep) 6

2. Практическая часть 6-7

Вывод 8

Листинг 1 «Новый timer.c» 9-15

ЦЕЛЬ РАБОТЫ

Цель работы – изучение системы управления процессами, а также механизма работы системного таймера в ОС Pintos, анализ его недостатков и модификация его алгоритма.

ХОД РАБОТЫ

1. Теоретическая часть

1.1 Что такое системный таймер?

Системный таймер — это специальное устройство компьютера, которое с равными интервалами времени генерирует сигналы-прерывания для процессора. Эти прерывания позволяют операционной системе «отсчитывать» время в тиках, измерять интервалы, организовывать переключение между процессами и реализовывать функции задержек и сна. По сути, системный таймер выступает в роли точного «метронома» ОС, благодаря которому она знает, сколько времени прошло и может управлять выполнением программ во времени.

В PintOS системный таймер реализован как программная обёртка над аппаратным таймером (в эмуляторе QEMU это PIT — Programmable Interval Timer), который генерирует прерывания с частотой, заданной макросом TIMER_FREQ (по умолчанию 100 раз в секунду). Каждое прерывание вызывает обработчик timer_interrupt, где увеличивается глобальный счётчик тиков и выполняются действия, зависящие от времени: обновление статистики, проверка, не истекло ли время сна у потоков, переданных в timer_sleep, а также вызов функций планировщика для переключения потоков при необходимости. Таким образом, в PintOS системный таймер служит источником «тактов» для ядра, позволяя ему измерять время и управлять выполнением процессов в соответствии с временными интервалами.

1.2 Функции системного таймера в pintOS.

Функция	Описание
void timer_init (void)	Настраивает аппаратный таймер, чтобы ОС получала прерывания раз в тик, и регистрирует функцию-обработчик этих прерываний.
void timer_calibrate(void)	Измеряет, сколько пустых циклов выполняется за один тик таймера, и сохраняет это число в глобальной переменной loops_per_tick.
int64_t timer_ticks (void)	Возвращает количество тактов таймера с момента загрузки ОС.
int64_t timer_elapsed (int64_t then)	Возвращает количество тиков, прошедших с момента then
void timer_sleep (int64_t ticks)	Прерывает текущий процесс примерно на ticks тактов таймера.
void timer_msleep (int64_t ms)	Прерывает текущий процесс примерно на миллисекунды ms.
void timer_usleep (int64_t us)	Прерывает текущий процесс примерно на микросекунды us.
void timer_nsleep (int64_t ns)	Прерывает текущий процесс примерно на наносекунды ns.
void timer_mdelay (int64_t ms)	Активная задержка примерно на ms миллисекунд; поток не блокируется, а крутится в цикле, «сжигая» CPU. Используется только для очень коротких пауз, где нельзя засыпать поток.
void timer_udelay (int64_t us)	Активная задержка примерно на us микросекунд; поток не блокируется, а крутится в

	цикле, «сжигая» CPU. Используется только для очень коротких пауз, где нельзя засыпать поток.
Void timer_ndelay (int64_t ns)	Активная задержка примерно на ns наносекунд; поток не блокируется, а крутится в цикле, «сжигая» CPU. Используется только для очень коротких пауз, где нельзя засыпать поток.
Void timer_print_stats (void)	Печатает статистику таймера. Выводит количество тактов, которое отведено под определенный процесс.
static void timer_interrupt (struct intr_frame *args UNUSED)	Вызывается на каждом тике таймера: увеличивает счётчик ticks, обновляет статистику потоков (thread_tick())
static bool too_many_loops (unsigned loops)	Проверяет, помещается ли выполнение loops пустых циклов (busy_wait) в один тик таймера.
static void NO_INLINE busy_wait (int64_t loops)	busy_wait(loops) — выполняет пустой цикл loops раз. Используется для активных задержек в PintOS.
static void real_time_sleep (int64_t num, int32_t denom)	Универсальная функция сна на num/denom секунд: если время > 0 тик, поток блокируется (timer_sleep), иначе выполняется активная задержка (real_time_delay).
static void real_time_delay (int64_t num, int32_t denom)	Активная задержка на num/denom секунд: выполняет цикл заданное количество раз, используя калиброванное значение loops_per_tick. Применяется для очень коротких пауз (меньше одного тика).

1.3 Блок-схема алгоритма работы функции timer_sleep()



2. Практическая часть

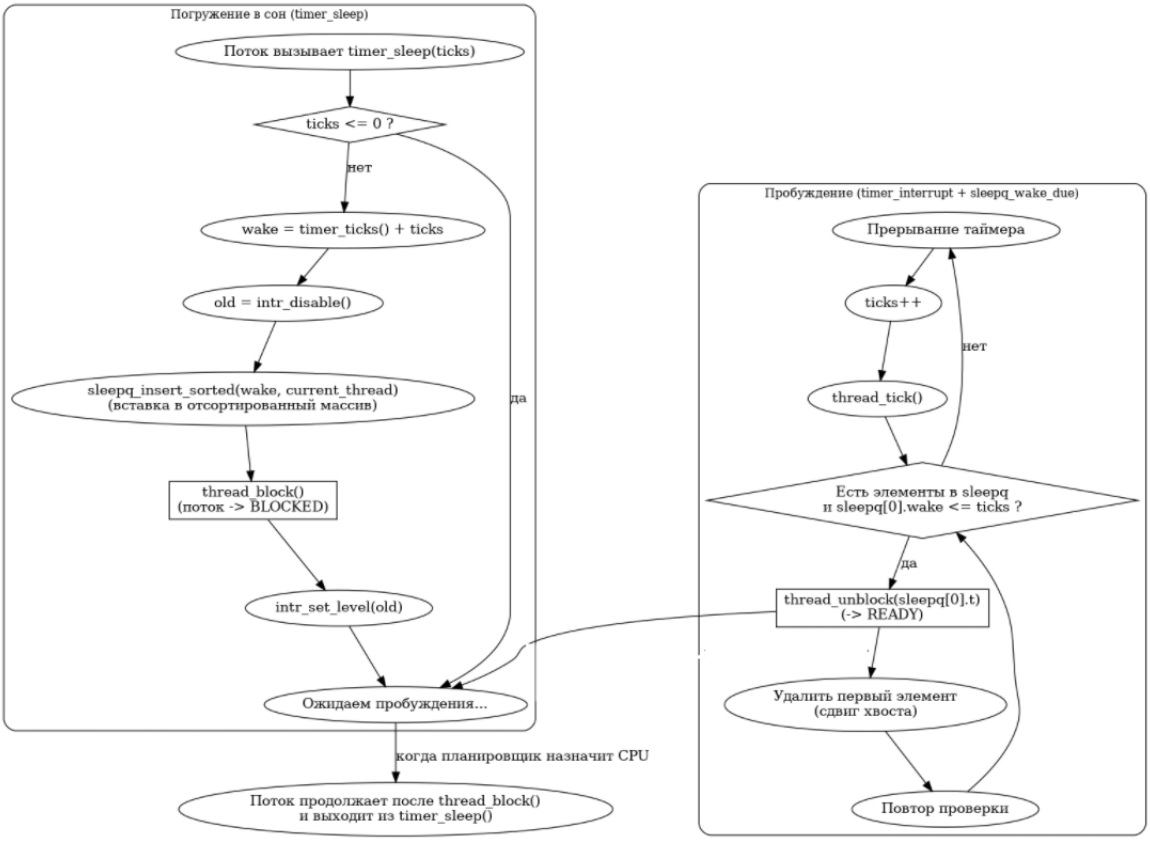
В ходе работы был разработан новый алгоритм функции timer_sleep(), позволяющий избавиться от активного ожидания. Теперь поток, вызвавший функцию, блокируется до наступления момента пробуждения. Для этого каждому потоку вычисляется время пробуждения в тиках таймера, после чего поток помещается в отсортированную очередь «спящих». Такая организация

гарантирует, что поток будет разбужен точно по истечении заданного времени, без дополнительного ожидания.

В отличие от предыдущей реализации, где использовался цикл с вызовом thread_yield(), поток больше не занимает процессорное время во время сна. Это значит, что другие процессы могут свободно выполняться, и «спящий» поток им не мешает. Разбудить его может только обработчик прерывания таймера, который на каждом тике проверяет начало очереди и вызывает thread_unblock() для тех потоков, чей срок пробуждения наступил.

Очередь спящих потоков реализована в виде массива структур, отсортированных по времени пробуждения. Вставка нового потока происходит в правильное место при помощи бинарного поиска, что сохраняет порядок элементов. Благодаря этому при пробуждении можно просто идти от начала массива до тех пор, пока время пробуждения не станет больше текущего значения таймера.

Блок схема нового алгоритма:



ВЫВОД

В ходе работы была модернизирована функция timer_sleep() в PintOS. Вместо активного ожидания потоков реализован механизм блокировки: каждый поток помещается в отсортированную очередь «спящих» с указанием момента пробуждения и переводится в состояние BLOCKED. В обработчике прерывания таймера потоки своевременно разблокируются и переводятся в состояние READY. Таким образом, удалось обеспечить точное время пробуждения процессов, исключить лишнюю загрузку процессора и повысить эффективность работы системы.

Листинг 1 «Новый timer.c»

```
#include "devices/timer.h"
#include <debug.h>
#include <inttypes.h>
#include <round.h>
#include <stdio.h>
#include "devices/pit.h"
#include "threads/interrupt.h"
#include "threads/synch.h"
#include "threads/thread.h"
#include "threads/malloc.h"
#include "threads/thread.h"
#include "threads/interrupt.h"
#include <string.h>
/* See [8254] for hardware details of the 8254 timer chip. */

#if TIMER_FREQ < 19
#error 8254 timer requires TIMER_FREQ >= 19
#endif
#if TIMER_FREQ > 1000
#error TIMER_FREQ <= 1000 recommended
#endif

struct sleep_entry{
     int64_t wake;        // тик, когда будить
     struct thread *t;    // кого будить
};

static struct sleep_entry *sleepq = NULL; // массив структур
sleep_entry
static size_t sleepq_cnt = 0;             // now count
static size_t sleepq_cap = 0;             // емкость

/* Number of timer ticks since OS booted. */
static int64_t ticks;

/* Number of loops per timer tick.
   Initialized by timer_calibrate(). */
static unsigned loops_per_tick;

static intr_handler_func timer_interrupt;
static bool too_many_loops (unsigned loops);
static void busy_wait (int64_t loops);
static void real_time_sleep (int64_t num, int32_t denom);
static void real_time_delay (int64_t num, int32_t denom);

static void sleepq_ensure_cap (size_t need){          // расширение
динамического массива спящих потоков
  size_t new_cap;                                      // во сколько
увеличить ёмкость
  struct sleep_entry *new_arr;                    // новый буфер

  if (need <= sleepq_cap) return;                     // если текущей
ёмкости хватает, ничего не делаем

  if (sleepq_cap != 0)
  {
```

```
    new_cap = sleepq_cap * 2;                       // удвоить ёмкость
  }

  else
  {
    new_cap = 64;                                          // начать с 64
элементов
  }

  while (new_cap < need) new_cap *=2;

  new_arr = malloc (new_cap * sizeof *new_arr);
  ASSERT (new_arr != NULL);

  if (sleepq !=NULL && sleepq_cnt > 0) {
    memcpy (new_arr, sleepq, sleepq_cnt * sizeof *new_arr);
    free (sleepq);
  }

  sleepq = new_arr;
  sleepq_cap = new_cap;
}

/* Вставка так, чтобы массив оставался отсортирован по wake (по
возрастанию) */
static void
sleepq_insert_sorted (int64_t wake, struct thread *t) {  // wake - тик
времени, когда нужно разбудить ( от меньшего к большему )
  size_t low, high, mid;
  /* low - нижняя граница поиска
     high - верхняя граница поиска
     mid - середина между low и high
      */
  sleepq_ensure_cap (sleepq_cnt + 1); // гарантируем, что в массиве
ещё есть место для одного элемента

  low = 0;
  high = sleepq_cnt;
  while (low < high) {
    mid = (low + high) / 2;
    if (sleepq[mid].wake <= wake) low = mid + 1;
    else high = mid;
  }

  /* сдвиг вправо начиная с хвоста до позиции low */
  size_t i = sleepq_cnt;
  while (i > low) {
    sleepq[i] = sleepq[i - 1];
    i--;
  }

  sleepq[low].wake = wake;
  sleepq[low].t    = t;
  sleepq_cnt++;
}

/* Разбудить всех, чей срок настал (wake <= now_ticks). */
static void
```

```
sleepq_wake_due (int64_t now_ticks) {
  size_t i = 0;

  /* Массив отсортирован по wake, поэтому идём с начала подряд. */
  while (i < sleepq_cnt && sleepq[i].wake <= now_ticks) {
    thread_unblock (sleepq[i].t);
    i++;
  }

  if (i == 0) return;               /* никого не будим */

  /* Сдвигаем оставшуюся часть к началу на i позиций. */
  if (i < sleepq_cnt) {
    size_t j = 0;
    while (i + j < sleepq_cnt) {
      sleepq[j] = sleepq[i + j];
      j++;
    }
  }
  sleepq_cnt -= i;
}

/* Sets up the timer to interrupt TIMER_FREQ times per second,
   and registers the corresponding interrupt. */
void
timer_init (void) // настройка таймера
{
  pit_configure_channel (0, 2, TIMER_FREQ);
  intr_register_ext (0x20, timer_interrupt, "8254 Timer");
  sleepq_cap = 64;
  sleepq_cnt = 0;
  sleepq = malloc(sizeof *sleepq * sleepq_cap);
  ASSERT(sleepq !=NULL);
}


/* Calibrates loops_per_tick, used to implement brief delays. */
void
timer_calibrate (void)           // Подбирает значение loops_per_tick,
нужно для точных, активных задержек
{
  unsigned high_bit, test_bit;

  ASSERT (intr_get_level () == INTR_ON);
  printf ("Calibrating timer...  ");

  /* Approximate loops_per_tick as the largest power-of-two
     still less than one timer tick. */
  loops_per_tick = 1u << 10;
  while (!too_many_loops (loops_per_tick << 1))
    {
      loops_per_tick <<= 1;
      ASSERT (loops_per_tick != 0);
    }

  /* Refine the next 8 bits of loops_per_tick. */
  high_bit = loops_per_tick;
```

```
  for (test_bit = high_bit >> 1; test_bit != high_bit >> 10; test_bit
>>= 1)
    if (!too_many_loops (high_bit | test_bit))
      loops_per_tick |= test_bit;

  printf   ("%'"PRIu64"   loops/s.\n",   (uint64_t)   loops_per_tick   *
TIMER_FREQ);
}

/* Returns the number of timer ticks since the OS booted. */
int64_t
timer_ticks (void)
{
  enum intr_level old_level = intr_disable ();
  int64_t t = ticks;
  intr_set_level (old_level);
  return t;
}

/* Returns the number of timer ticks elapsed since THEN, which
   should be a value once returned by timer_ticks(). */
int64_t
timer_elapsed (int64_t then)
{
  return timer_ticks () - then;
}

/* Sleeps for approximately TICKS timer ticks.  Interrupts must
   be turned on. */
void
timer_sleep (int64_t ticks)
{
  if (ticks <=0) return;

  int64_t wake = timer_ticks() + ticks;
  enum intr_level old = intr_disable ();

  sleepq_insert_sorted (wake, thread_current());
  thread_block ();

  intr_set_level(old);
}

/* Sleeps for approximately MS milliseconds.  Interrupts must be
   turned on. */
void
timer_msleep (int64_t ms)
{
  real_time_sleep (ms, 1000);
}

/* Sleeps for approximately US microseconds.  Interrupts must be
   turned on. */
void
timer_usleep (int64_t us)
{
  real_time_sleep (us, 1000 * 1000);
}
```

```
/* Sleeps for approximately NS nanoseconds.  Interrupts must be
   turned on. */
void
timer_nsleep (int64_t ns)
{
  real_time_sleep (ns, 1000 * 1000 * 1000);
}

/* Busy-waits for approximately MS milliseconds.  Interrupts need
   not be turned on.

   Busy waiting wastes CPU cycles, and busy waiting with
   interrupts off for the interval between timer ticks or longer
   will cause timer ticks to be lost.  Thus, use timer_msleep()
   instead if interrupts are enabled. */
void
timer_mdelay (int64_t ms)
{
  real_time_delay (ms, 1000);
}

/* Sleeps for approximately US microseconds.  Interrupts need not
   be turned on.

   Busy waiting wastes CPU cycles, and busy waiting with
   interrupts off for the interval between timer ticks or longer
   will cause timer ticks to be lost.  Thus, use timer_usleep()
   instead if interrupts are enabled. */
void
timer_udelay (int64_t us)
{
  real_time_delay (us, 1000 * 1000);
}

/* Sleeps execution for approximately NS nanoseconds.  Interrupts
   need not be turned on.

   Busy waiting wastes CPU cycles, and busy waiting with
   interrupts off for the interval between timer ticks or longer
   will cause timer ticks to be lost.  Thus, use timer_nsleep()
   instead if interrupts are enabled.*/
void
timer_ndelay (int64_t ns)
{
  real_time_delay (ns, 1000 * 1000 * 1000);
}

/* Prints timer statistics. */
void
timer_print_stats (void)
{
  printf ("Timer: %"PRId64" ticks\n", timer_ticks ());
}

/* Timer interrupt handler. */
static void
timer_interrupt (struct intr_frame *args UNUSED)
```

```
{
  ticks++;
  thread_tick ();

  /* Разбудить тех, чей дедлайн наступил на этом тике. */
  sleepq_wake_due (ticks);
}

/* Returns true if LOOPS iterations waits for more than one timer
   tick, otherwise false. */
static bool
too_many_loops (unsigned loops)
{
  /* Wait for a timer tick. */
  int64_t start = ticks;
  while (ticks == start)
    barrier ();

  /* Run LOOPS loops. */
  start = ticks;
  busy_wait (loops);

  /* If the tick count changed, we iterated too long. */
  barrier ();
  return start != ticks;
}

/* Iterates through a simple loop LOOPS times, for implementing
   brief delays.

   Marked NO_INLINE because code alignment can significantly
   affect timings, so that if this function was inlined
   differently in different places the results would be difficult
   to predict. */
static void NO_INLINE
busy_wait (int64_t loops)
{
  while (loops-- > 0)
    barrier ();
}

/* Sleep for approximately NUM/DENOM seconds. */
static void
real_time_sleep (int64_t num, int32_t denom)
{
  /* Convert NUM/DENOM seconds into timer ticks, rounding down.

        (NUM / DENOM) s
     ---------------------- = NUM * TIMER_FREQ / DENOM ticks.
     1 s / TIMER_FREQ ticks
  */
  int64_t ticks = num * TIMER_FREQ / denom;

  ASSERT (intr_get_level () == INTR_ON);
  if (ticks > 0)
    {
      /* We're waiting for at least one full timer tick.  Use
         timer_sleep() because it will yield the CPU to other
```

```
         processes. */
      timer_sleep (ticks);
    }
  else
    {
      /* Otherwise, use a busy-wait loop for more accurate
         sub-tick timing. */
      real_time_delay (num, denom);
    }
}

/* Busy-wait for approximately NUM/DENOM seconds. */
static void
real_time_delay (int64_t num, int32_t denom)
{
  /* Scale the numerator and denominator down by 1000 to avoid
     the possibility of overflow. */
  ASSERT (denom % 1000 == 0);
  busy_wait (loops_per_tick * num / 1000 * TIMER_FREQ / (denom /
1000));
}
```